SQL - Revision Notes

1. The shape of a SELECT query

SQL is declarative: you describe the result you want, not the steps to compute it. A single table query has the form SELECT columns FROM table WHERE condition GROUP BY columns HAVING condition ORDER BY columns LIMIT count.

The logical processing order is FROM, then WHERE, then GROUP BY, then HAVING, then SELECT, then ORDER BY, then LIMIT. This order matters. An alias defined in SELECT cannot be used in WHERE, but it can be used in ORDER BY.

2. Filtering with WHERE

WHERE keeps only the rows that satisfy a condition and it is applied before grouping. Useful operators are =, <>, <, <=, >, >=, BETWEEN, IN, LIKE, IS NULL and EXISTS.

LIKE supports the wildcards % for any sequence of characters and _ for a single character. To search for a literal percent sign you escape it as \%.

The three valued logic of SQL matters. Comparisons with NULL produce UNKNOWN, not true or false. WHERE only returns rows where the condition is TRUE, so rows where the condition is UNKNOWN are dropped. This is why "column <> 'value'" will not find NULLs and you need "column IS NULL OR column <> 'value'".

3. Joins

An inner join keeps only the rows that match on both sides. A left outer join keeps every row from the left table and adds NULL for unmatched right rows, which is the join to use when you want all students even if they have not enrolled in any course. A right outer join is the mirror image and a full outer join keeps rows from both sides.

Self joins join a table to itself, usually through a manager_id style foreign key. Cross joins produce every combination and are used deliberately in some reporting queries.

Joining more than two tables is best written one join at a time in order, and using table aliases keeps the query readable.

4. Aggregation and grouping

Aggregate functions compute one value from many rows: COUNT, SUM, AVG, MIN and MAX. COUNT(column) ignores NULL values while COUNT(*) counts rows, so they can differ.

GROUP BY collapses rows that share the same values in the grouping columns and applies the aggregate functions to each group. HAVING filters those groups after aggregation, and it can reference aggregate functions, while WHERE cannot.

The usual exam question is the one about order of clauses: WHERE is applied before grouping and HAVING after, so filtering on an aggregate such as HAVING COUNT(*) > 5 cannot be written in WHERE.

5. Subqueries

A subquery is a query nested inside another query. It returns one value, one row, or many rows, and that shape decides where it can be used: a single value subquery can be compared with =, a multi row subquery needs IN, and EXISTS tests whether the subquery returns anything at all.

Correlated subqueries reference the outer query, which means the inner query runs once per outer row. That is convenient to write and often slow to execute.

6. Views and indexes for queries

A view is a named query stored in the database. Views can restrict the columns a user sees, hide complexity, and present the same data in a stable way. Updates through a simple view are allowed, but updates through a view that uses aggregation or joins are usually restricted.

When a query is slow, first check whether the filtered and joined columns have indexes. EXPLAIN shows the plan the optimizer chose and whether it did a full table scan.